

Relatório Final — Especificação v3 implementada (Fases 0-7)

Projeto: LangNet · caso-teste ClinIA · **Data:** 2026-08-14 · **Executor:** Claude **Base:** ESPEC-SPECIFICATION-ENGINEERING-v3.md + PLANO-IMPLEMENTACAO-v3.md

Este documento fecha o plano v3: as **8 fases (0-7)** foram implementadas, **validadas** (com regressão zero verificada por smoke-test E2E a cada fase) e **commitadas/pushadas** com o protocolo de **checkpoint-antes-de-cada-fase** para rollback seguro. Abaixo: a Fase 7, a consolidação das 8 fases, a trilha de commits e as ressalvas honestas.

1. Fase 7 — Log de suposições & limitações (Inserção D)

O que foi feito. `_emit_okf_bundle` passa a emitir `knowledge/assumptions.md` (conceito OKF type: Assumptions & Limitations) com as **suposições** do app gerado (schema é fonte de verdade; escritas pela camada determinística; contexto é dado não comando; saída sob contrato+pós-condições; LLM local pode variar) e as **limitações derivadas programaticamente** (tasks agênticas que dependem do LLM; tasks sem contrato/verificação; descrições de tabela genéricas). `log.md` aponta para ele.

Validação (): `assumptions.md` emitido e conformante; lista 5 suposições + limitações reais (12 tasks dependem do LLM; `visualizar_kpis` sem contrato; descrições genéricas). **E2E smoke VERDE** (regressão zero).

Benefício: auditabilidade — suposições implícitas viram registro explícito e rastreável (passo 6 do workflow), fechando o ciclo `especificação → geração → validação → revisão → auditoria`.

2. Consolidação — as 8 fases

Fase	Inserção	O que entrega	Validação-chave	Benefício
0	Harness	<code>tools/regen/</code> (regen + smoke + services)	regen do zero + smoke VERDE	ciclo reproduzível; regressão zero verificável
1	A	Contrato de saída (JSON Schema) no ws-server	fault-injection → fail-loud; <code>nivel_confianca</code> numérico	mata bugs de saída (<code>{raw}</code> , enum↔float, NOT NULL)
2	E	Bundle OKF de contexto (agentes aterrados)	contexto = tabelas reais; zero <code>historico_medico</code>	ataca alucinação na raiz
3	G	Cadeia de comando / hierarquia de instruções	red-team: injeção ignorada	fecha o flanco de prompt-injection
4	B	Verificação/pós-condições por task	negativo → erro claro sem persistir; <code>row_check</code>	barra o "plausível mas errado"
5	F	Proveniência OKF + Attested Computation	<code>generated/verified</code> (trust tier); 12 conceitos AC	rastreabilidade padrão e portátil
6	C	Gate de requisito + auto-crítica	gate aponta lacunas; <code>quality_report.md</code>	maior ROI (requisito upstream)
7	D	Log de suposições & limitações	<code>assumptions.md</code> conformante	auditabilidade

As 3 famílias de dor resolvidas na fonte: - **Saída torta** ({raw} , enum↔float, campo faltando) → **contrato A** (+ aposenta parseAgentResult / _cv). - **Alucinação** (inventar tabela/entidade) → **contexto OKF E** — com o flanco de **injeção fechado por G**. - **Persistência errada** (FK nula / registro cruzado / plausível-mas-errado) → **verificação B**.

3. Trilha de commits (checkpoint-antes-de-cada-fase → selo → relatório)

```
F0 d1deb7c (ANTES) → e7b595f (selo) → 0608a79 (rel. F0/F1)
F1 a69a0c3 (ANTES) → c0c4b84 (selo) → b5df72b (rel.)
F2 0fcf526 (ANTES) → f5d3933 (selo) → 2e16de4 (rel.)
F3 620ba41 (ANTES) → d8f8ff5 (selo) → 16d272d (rel.)
F4 a3bab30 (ANTES) → 47e697f (selo) → d3d0027 (rel.)
F5 cc6f806 (ANTES) → 507b02f (selo) → 1424576 (rel.)
F6 fcf4ad4 (ANTES) → 5b54ad5 (selo) → 7227724 (rel.)
F7 99f5172 (ANTES) → f646796 (selo)
```

Cada **checkpoint** descreve, no texto do commit, o que a fase implementaria (rollback seguro). Todos pushados para `origin/master`.

4. Provas globais (critérios do plano — todos)

1. **Saída sob contrato (A)**: nenhuma task devolve {raw}; incompletude → `error` explícito.
 2. **Contexto aterrado (E)**: agentes usam o bundle OKF; regressão do `historico_medico` eliminada.
 3. **Injeção fechada (G)**: comando malicioso em campo/contexto é ignorado (red-team).
 4. **Verificação ativa (B)**: FK nula / falta de contexto barradas antes de persistir.
 5. **Rastreabilidade OKF (F)**: trust tier (`generated / verified`) + Attested Computation.
 6. **Requisito com gate (C)**: spec incompleta é apontada; auto-crítica disponível.
 7. **Auditoria (D)**: suposições/limitações registradas.
 8. **Regressão zero**: o fluxo clínico E2E (triagem→pré-diagnóstico→encaminhamento→prontuário→consulta) persistiu a cadeia ligada **em todas as 8 fases** (smoke VERDE).
-

5. O harness (Fase 0) — `tools/regen/`

Versionado, reproduzível: `regen.sh [all|screens|adapters|wsserver|okf]`, `services.sh [up|down|status]`, `smoke.sh` (E2E + verificação no banco), `fault_inject.py` (contrato), `red_team.py` (injeção), `verify_chain.py`. Resolve o scratchpad volátil e dá **regressão zero verificável por um comando**.

6. Ressalvas honestas

- **LLM local** lento/às vezes flaky (o smoke e os testes usam retry; o contrato/checks mitigam).
 - **Reforço de prompt (C)** afeta a geração de spec de **todos os projetos** (rota compartilhada — mudança aditiva); o efeito pleno (tasks já com constraints/edge_cases) aparece ao **gerar uma spec nova**.
 - **Proveniência OKF (F)**: o `verified` humano depende do "Aprovar" na pipeline; a ClinIA está `draft` (unverified) — comprovamos a lógica do trust tier com um caso aprovado.
 - **Descrições de tabela** no bundle são genéricas — um passe de *Enrichment* as melhoraria (registrado em `assumptions.md`).
-

7. Conclusão

A **Especificação v3** — *specification engineering* + OKF + comportamento de agente — está **implementada e validada de ponta a ponta** no gerador do LangNet, com a **ClinIA** como caso-teste. As três famílias de bug que mais custaram (saída torta, alucinação, persistência errada) foram resolvidas **na fonte**, com o **flanco de injeção fechado, rastreabilidade padrão OKF, gate de requisito e auditoria** — tudo com **regressão zero verificável e rollback por fase**. Próximos passos opcionais (não do plano): regenerar a ClinIA por uma spec nova (para materializar constraints/edge_cases via o reforço C) e um passe de *Enrichment* das descrições de tabela.